

Final Research/Programming Assignment

Juliana Ng

MSDS 458 SEC 56

March 16, 2025

Abstract

This paper explores the performance of Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) at image data of face shape classification of female celebrities. Several regularization techniques are implemented to understand their effects on model development.

This research uses the *Face Shape Dataset* containing 5,000 color images across 5 face shapes - heart, oblong, oval, round, and square. TensorFlow and Keras are used to build the models. The best performing model of this research has 3 convolutional / max pooling layers of different kernel and stride sizes, L2 regularization, and a dense layer (256 nodes). It has 53% training accuracy and 38% testing accuracy. The loss curves show an underfitting model and confusion matrix shows an accurate classification of images to their labeled classes.

Introduction

The *Face Shape Dataset* is a labeled dataset collected by Niten Lama (Niten). It contains 5,000 images in 5 classes, with 1,000 images in each class. The classes are heart, oblong, oval, round, and square. In this paper, the dataset is used to train Deep Neural Networks (DNN) and Convolutional Neural Networks (CNN) to find the best model by conducting a series of experiments adjusting the hidden layers, hyperparameters, and various regularization techniques. Regularization techniques are employed to minimize overfitting of models. Data augmentation technique is used to add diversity to the already existing data by creating modified versions of them (DataCamp, 2024). It improves accuracy and prevents overfitting of models. SGD optimizer is used because of its ability to generalize better than adam to improve model accuracy (Ali, 2021). We would like to train a model with high accuracy, minimal loss across training, validation, and testing stages and correctly classify the images with the labeled category.

Literature review

Pratch-yani on Github has developed a machine learning model for classifying face shapes using the same dataset. They used Multi-Task Cascaded Convolutional Neural Network (MTCNN) to automatically detect the face in the pre-processing stage and used data augmentation. The models were built from scratch and using pre-trained model VGG-16 architecture with pre-trained weights and achieved accuracy around 80% and 90% respectively. The oval face shape is the most often misclassified, mostly Asian oval shape misclassified as round shape (Pratch-yani, 2020).

Methods

This research is conducted using the *Face Shape Dataset* by Niten Lama (Niten). The dataset consists of 5000 images of female celebrities across 5 face shapes: heart, oblong, oval, round, and square. Each shape consists of 1000 images, 640 for training, 160 for validation and 200 for testing. The entire dataset is split into 3,200 images for training, 800 for validation, and 1,000 for testing. The main libraries are Tensorflow (v 2.18.0), Keras (v 3.8.0), matplotlib, and scikit-learn libraries. In the experiments, the architecture, regularization method, data augmentation level, optimizer and learning rate, batch size and epoch are tweaked to find a model with the best accuracy and RMSE, and correct classification of the images with the labelled class with high F1 score.

All models run on *Adam* optimizer have learning rate 0.0001 and clip value 1. Only experiment 9 is run on SGD optimizer with learning rate 0.01 and momentum 0.9 to understand the effects on the model's performance. Experiments 1-8, 10-12 are run for 10 epochs with batch size 16, while

experiment 13 are run with batch size 32. Experiments 14 and 15 are run for 20 epochs with batch size 64 and experiment 16 is run for 20 epochs with batch size 32. All models use ReLu activation function and softmax in the output layer. All model losses are measured with Sparse Categorical Cross Entropy. The models in each experiment is adjusted based upon the performance of the previous models.

Experiment 1 is the only DNN with 5 dense layers. Experiments 2-4 are CNNs with 5 layers. They differ in the kernel and stride sizes in the Conv2D and Maxpool2D layers. Experiments 5-7 are variations of experiments 2-4 using some data augmentation (DA1). The data augmentation includes 25 degree rotation, height, .25 width, and zoom range, horizontal flip, fill with nearest pixel, and 25 batch size. Experiment 8 has the same architecture as experiment 2, similar to experiment 5 but with a less data augmentation (DA2), - 10 degree rotation, height, .1 width, and zoom range, horizontal flip, fill with nearest pixel, and 10 batch size. Experiment 9 is a variation of experiment 8 ran with SGD optimizer.

Experiment 10 is a CNN with 7 layers, 2 additional layers than the previous models, and uses data augmentation (DA2). Experiment 11 is a variation of experiment 10 without using data augmentation. Experiment 12 is a CNN with a dropout layer 0.3 and uses L2 regularizer 0.01. Experiment 13 is a variation of experiment 12 run with batch size of 32. Experiment 14 is a CNN with 12 layers. There is a dropout layer 0.3 after every Maxpool2D layer. Experiment 15 is a variation of experiment 12 but uses 0.5 in the dropout layer and has a batch size of 64. Experiment 16 is a model from Github using the dataset where the author achieved 77% accuracy for the validation dataset (Pratch-yani, 2020).

The images in this dataset are first resized to 64 x 64 pixels while keeping the aspect ratio by padding the image with white background, then rescaled to between 0 - 1 by dividing them by 255. The accuracy and loss for training, validation, and testing, confusion matrix are calculated and visualized with matplotlib. The classification report is constructed using scikit-learn.

Results

Exp	NN type (DNN/CNN)	Architecture	Optimizer, epochs, batch size	Train acc	Val acc	Test acc	Train loss	Val loss	Test loss
1	D	5 dense layers: relu (512, 512, 256); softmax dense (5)	Optimizer 1	.426	.357	.363	1.364	1.472	1.571
2	C	5 layers: relu Conv2D(filter 128, kernel 5, 2 stride), Maxpool2D(size 2x2, stride 2), flatten, dense (256); softmax dense (5)		0.709	0.853	.364	0.386	1.566	1.540
3	C	5 layers: relu Conv2D(filter 128, kernel 7, 2 stride), Maxpool2D(size 2x2, stride 2), dense (256); softmax dense (5)		.669	.396	.353	.923	1.550	1.572
4	C	5 layers: relu Conv2D(filter 128, kernel 5, 3 stride), Maxpool2D(size 2x2, stride 3), flatten, dense (256); softmax dense (5)		.458	.351	.383	1.334	1.477	1.497
5	C	Experiment 2 architecture + DA1		.237	.245	.263	1.599	1.588	1.635
6	C	Experiment 3 architecture + DA1		.265	.316	.312	1.580	1.570	1.610
7	C	Experiment 4 architecture + DA1		.244	.241	.295	1.588	1.584	1.601
8	C	Experiment 2 architecture + DA2		.337	.315	.345	1.496	1.529	1.487
9	C	Experiment 8 with SGD optimizer	SGD Optimizer	.309	.334	.327	1.537	1.531	1.538
10	C	7 layers: relu Conv2D(filter 128, kernel 5, 1 stride), Maxpool2D(size 2x2, stride 1), Conv2D(filter 128, kernel 5, 1 stride), Maxpool2D(size 2x2, stride 2, flatten, dense (256); softmax dense (5) + DA2	Optimizer 1	.382	.361	.374	1.445	1.471	1.449
11	C	Experiment 10 without data augmentation		.895	.351	.351	.328	2.786	1.525
12	C	8 layers: relu Conv2D(filter 128, kernel 3, 1 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 1), dropout (.3), Conv2D(filter 128, kernel 5, 1 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 2), flatten, dense (256); softmax dense (5)		.638	.377	.369	1.150	1.728	1.721

13	C	Experiment 12, batch size 32	Optimizer 1, batch size 32	.572	.363	.359	1.324	1.722	1.787
14	C	12 layers: relu Conv2D(filter 128, kernel 3, 1 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 1), dropout (.3), Conv2D(filter 128, kernel 5, 1 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 1), dropout (.3), Conv2D(filter 128, kernel 5, 2 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 2), dropout (.3), flatten, dense (256); softmax dense (5)	Optimizer 2	.534	.399	.383	1.528	1.800	1.773
15	C	Experiment 12, dropout .5, batch size 64		.526	.380	.361	1.412	1.668	1.709
16	C	Model from Github, 17 layers: relu Conv2D(filter 32, kernel 3, 1 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 1), dropout (.3), Conv2D(filter 64, kernel 3, 1 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 1), dropout (.3), Conv2D(filter 128, kernel 3, 1 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 1), dropout (.3), Conv2D(filter 256, kernel 3, 1 stride, L2 regularizer .01), Maxpool2D(size 2x2, stride 1), dropout (.3), flatten, dense (256, 128), dropout (.5); softmax dense (5) + DA3	Optimizer 3	.311	.279	.351	2.594	2.613	2.572

Data augmentation 1 (DA1): rotation 25, height/width/zoom range .25, horizontal flip, fill with nearest pixel, batch size 25

Data augmentation 2 (DA2): rotation 10, height/width/zoom range .1, horizontal flip, fill with nearest pixel, batch size 10

Data augmentation 3 (DA3): rotation 20, horizontal flip, batch size 5

Optimizer 1: Adam, learning rate 0.0001, clip value 1, 10 epochs, batch size 16

Optimizer 2: Adam, learning rate 0.0001, clip value 1, 20 epochs, batch size 64

Optimizer 3: Adam, learning rate 0.0001, clip value1, 20 epochs, batch size 32

Experiment 1 - The only Deep Neural Network (DNN) recorded in this research as it is one of the better performing models compared to undocumented experiments, which weren't recorded due to the poor performance. The training, validation, and testing

accuracies ranged from 36% - 42%, losses 1.36 - 1.57, RMSE 1.86. The accuracy and loss curves show some overfitting and the confusion matrix and F1 scores show good prediction for heart, oblong, and square face shapes. The performance isn't great but better than most other models using Convolutional Neural Network (CNN).

Experiment 2 - This model has the best training and validation accuracies out of all models at 71% and 85% respectively, but testing accuracy is much lower at 36%. The overfitting is also shown in the accuracy and loss curves. The losses range from 0.39 - 1.5, RMSE 1.67. The confusion matrix doesn't show good predictions for face shapes except for oblong, which has the highest F1 score of 0.47.

Experiment 3 - Similar model as experiment 2 but has a kernel size 7. The training and validation accuracies dropped to 67% and 40% but closer to testing accuracy at 35%. Losses range from .92 - 1.57. Model is as overfitting as experiment 2, and a good prediction for oblong shape.

Experiment 4 - Similar results as experiment 3 when compared to experiment 2. It has a similar model to experiment 2 with stride 3. The training accuracy dropped to 46%, validation 35%, testing 38%, losses range from 1.33 - 1.50. The accuracy and loss curves show less of an overfitting as the previous 2 models but are still overfitting. It has a good F1 score for oblong shape and better F1 score for round and square shapes compared to the previous 2 models.

Experiment 5 - 7 - These models have the same architecture as experiment 2 - 4 but use moderate data augmentation. They all have very bad accuracies 24% - 31% and similar losses as the other models, with experiment 6 having the best accuracy out of these 3 models. The accuracy and loss curves look very bad and all models have either validation or testing accuracies higher than the training accuracy. Confusion matrices show class imbalance in the data that is probably due to data augmentation. All three models have good F1 score for oblong shape, Experiment 6 and 7 have good F1 score for square shape.

Experiment 8 - This model has the same architecture as experiment 2 but uses data augmentation. This is similar to experiment 5 but the data augmentation is less aggressive. It has lower training and validation accuracy than experiment 2 and the model isn't overfitting like experiment 2. The accuracy and loss curves show that the model is actually underfitting. The overall accuracy is better than experiment 5 35% vs 26% but they have similar loss values. The confusion matrix shows class imbalance in predictions and the model has good F1 scores for oblong, round, and square shapes.

Experiment 9 - This model has the same architecture as experiment 8 but uses SGD optimizer. It has similar performance metrics as experiment 8 but confusion matrix shows less of a class imbalance. The accuracy and loss curves show an underfitting model and the curves are very jagged and not smooth.

Experiment 10 - This model is similar to experiment 4 but it has 2 additional layers than all previous CNN models and uses the less aggressive data augmentation. It has similar performance metrics as experiment 4 but the curves are not smooth. The model has good F1 scores for all shapes except for oval shape, better than experiment 4.

Experiment 11 - This model is the same as experiment 10 without data augmentation. Similar to models without data augmentation, the training accuracy (90%) is a lot better than validation (35%) and testing accuracies (35%), showing a very overfitting model. This model has the best training accuracy out of all models. The model has a good F1 score for oblong and round shapes.

Experiment 12 - This model has 2 convolutional and 2 maxpool layers with a 0.3 dropout layer without using data augmentation. It also uses L2 regularizer 0.01. It has a decent training accuracy at 64% compared to all other models but validation and testing accuracies remain under 40%. The losses range from 1.2 to 1.7, RMSE 1.8. The accuracy and loss show some overfitting but a lot less than other overfitting models. It also has a good F1 score for all shapes except for oval.

Experiment 13 - This model is the same as experiment 12 but the batch size is increased to 32. It has similar performance metrics as experiment 12 but less overfitting. It doesn't have an F1 score as good as experiment 12.

Experiment 14 - This model has the most layers out of all models. It has 3 convolutional, 3 maxpool, and 3 dropout layers, and uses L2 regularizer 0.01. The model is run for 20 epochs and batch size is higher than other models at 64. It has similar performance metrics as experiment 13 but shows some underfitting. The accuracy and loss curves are smooth and show good fitting. It has a good prediction for the correct face shapes along the diagonal of the confusion matrix.

Experiment 15 - This model has the same architecture as experiment 12 but uses 0.5 dropout and runs on batch size of 64 for 20 epochs. It has better overall accuracy (39%) and RMSE (1.63) than experiments 12 (37%; 1.74) and 13 (36%, 1.68). The accuracy and loss curves show a good fitting model and it has good F1 scores for round and square shapes.

Experiment 16 - This model is found on Github from someone who has developed a model for the same dataset. It has the most layers out of all models, uses regularization techniques, and data augmentation. Their training and validation accuracies are 80% and 77%, however mine are 31% and 28%. It has the highest loss and the confusion matrix shows misclassification of face shapes and class imbalance, although it has a high F1 score for round shape.

Conclusions

While it is possible to achieve up to 90% training accuracy (experiment 11) and 85% validation accuracy (experiment 2), none of the models was able to achieve good training accuracy without an overfitting model. All models have testing accuracy below 40%, ranging from 26% - 38% and high RMSE around 1.7. I don't think any of the models are good enough for doing face shape predictions since they either have low accuracy or high accuracy in training but low accuracy in testing. Nevertheless, the better model in this research is experiment 14, it has a good F1 score for all face shapes compared to other models. The training accuracy isn't great at 53% and validation/testing accuracy at 40% and 38%, but it doesn't show overfitting like other models. It is slightly underfitting, which should be addressed in future experiments. Oblong shape is the easiest shape to predict in all models, followed by square and round shapes. Heart is often confused for oval and oval is the hardest shape to predict.

The discrepancy in model performance in experiment 16 and that of Pratch's may be due to the method of image preprocessing and model runtime. The images in the research were resized to 64 x 64 keeping the aspect ratio and Pratch's were 224 x 224. Most of the models were run for 10 epochs and 16 batch size and Pratch's were 150 epochs and 32 batch size. Many unrecorded experiments were performed with larger image size and batch size but they had horrible accuracy and often the loss in the first epoch of training results in NaN (not a number) values, which is probably due to the lack of computational power.

The model architecture, regularization techniques, and batch size have significant impact on model performance, often improving the metrics the more complex it gets. SGD optimizer

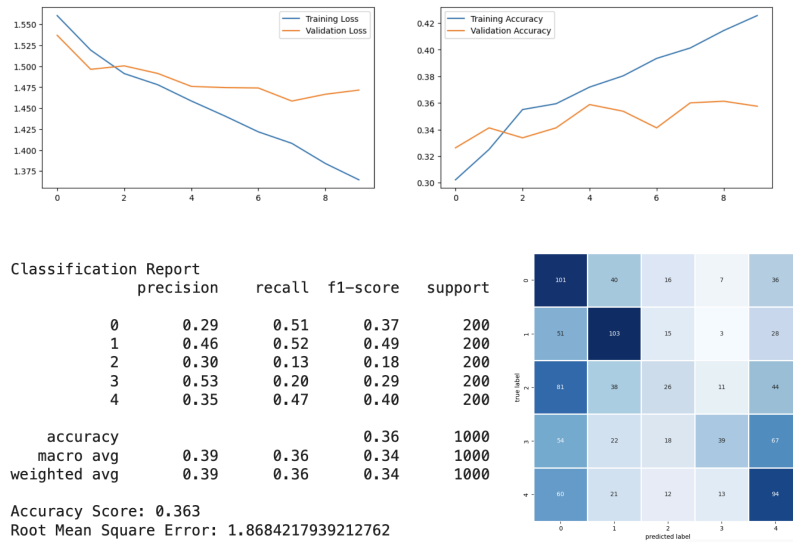
(compared to adam) and epochs did not have much effect on performance as the model stops improving/training (patience 3) before reaching the designated epoch number. Data augmentation often led to dataset imbalance in different classes shown in the confusion matrix, though improving validation accuracy. More aggressive data augmentation (DA1) returned worse performance than mild data augmentation (DA2). Future experiments need to address this issue and finetune the parameters of data augmentation to result in better model fit.

This research demonstrates Convolutional Neural Networks (CNN) have the ability to classify face shape images to multiple classes. We also learn that including regularization techniques, increasing model complexity, and increasing batch size of CNNs can minimize the overfitting problem of a model and increase accuracy. In this research, there is a huge challenge between improving model accuracy while keeping the model fit optimal. In the future, pre-trained models for facial recognition can be implemented in the models and larger image sizes can be used to train models with higher batch size and epochs when more computational power is available.

Appendix

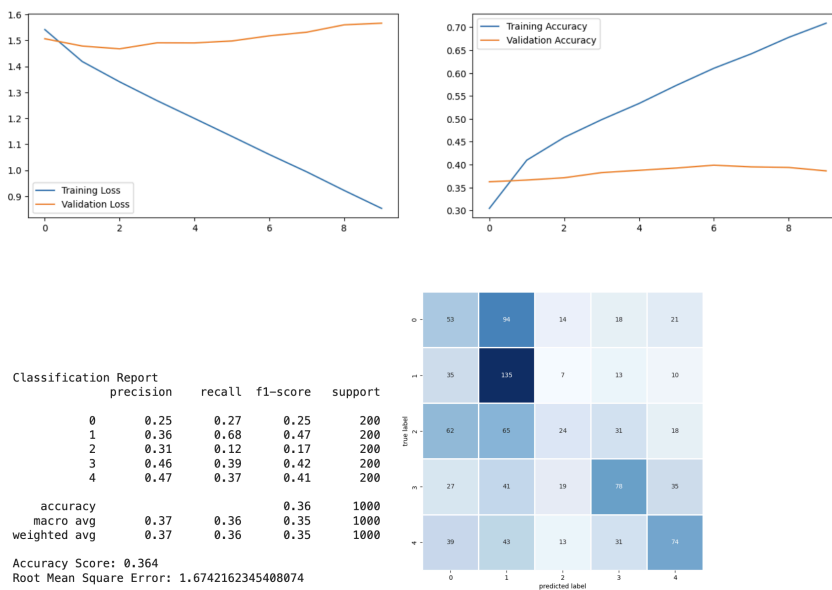
1. Experiment 1: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



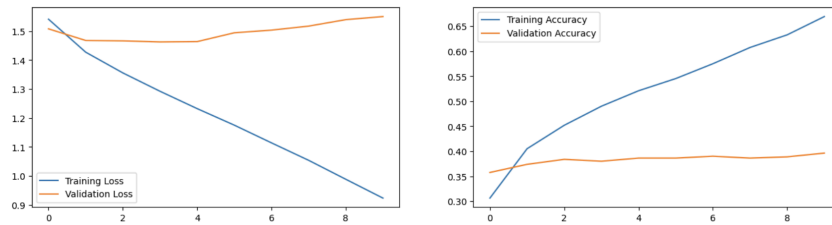
2. Experiment 2: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



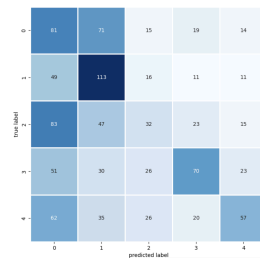
3. Experiment 3: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



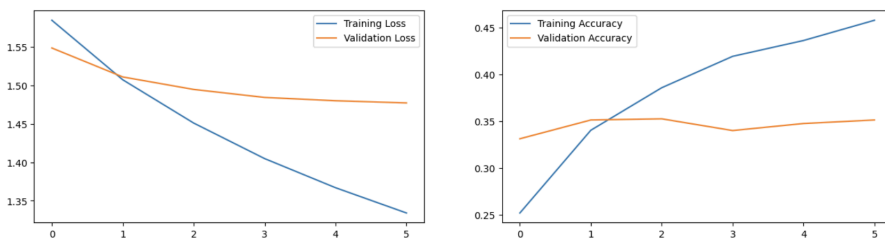
Classification Report				
	precision	recall	f1-score	support
0	0.25	0.41	0.31	200
1	0.38	0.56	0.46	200
2	0.28	0.16	0.20	200
3	0.49	0.35	0.41	200
4	0.47	0.28	0.36	200
accuracy			0.35	1000
macro avg	0.37	0.35	0.35	1000
weighted avg	0.37	0.35	0.35	1000

Accuracy Score: 0.353
Root Mean Square Error: 1.8041618552668715



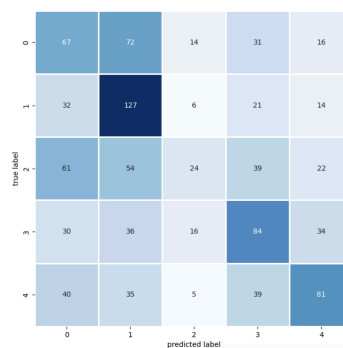
4. Experiment 4: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix

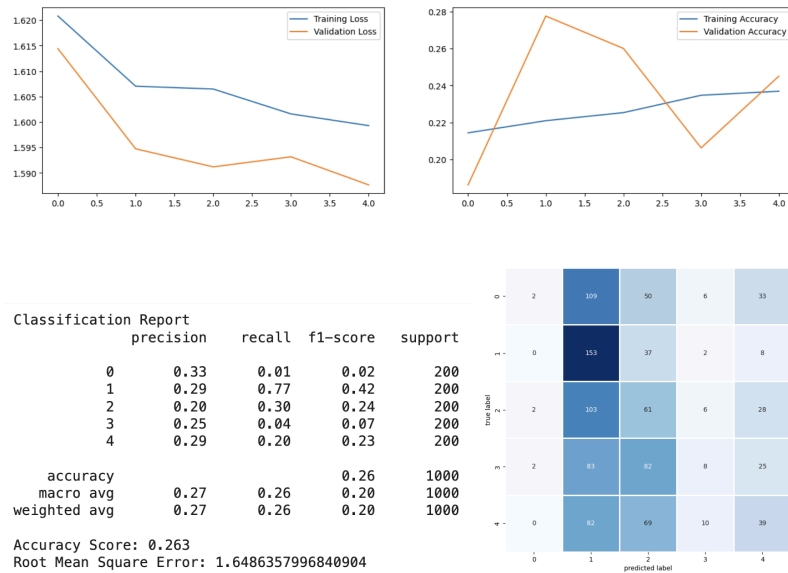


Classification Report				
	precision	recall	f1-score	support
0	0.29	0.34	0.31	200
1	0.39	0.64	0.48	200
2	0.37	0.12	0.18	200
3	0.39	0.42	0.41	200
4	0.49	0.41	0.44	200
accuracy			0.38	1000
macro avg	0.39	0.38	0.36	1000
weighted avg	0.39	0.38	0.36	1000

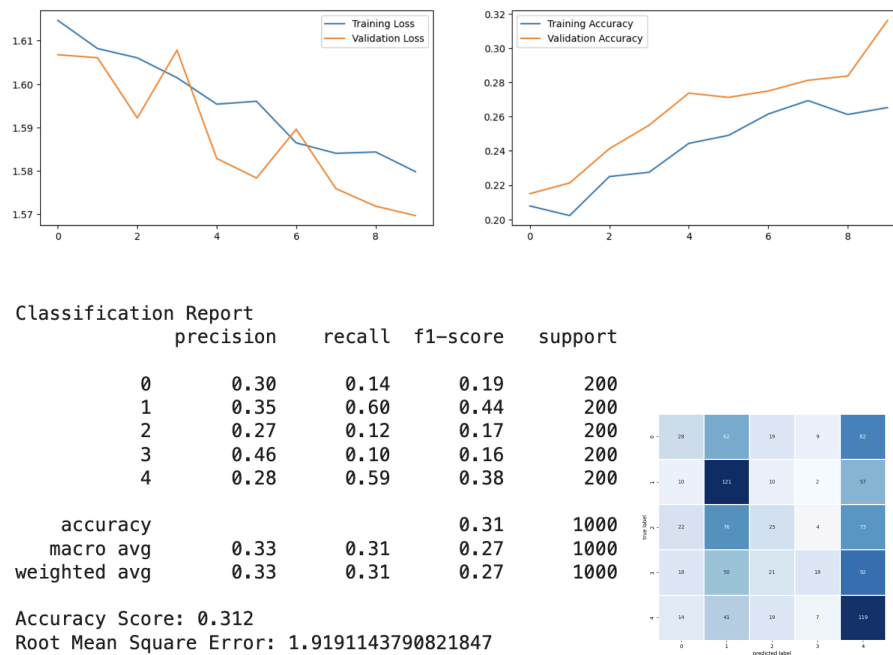
Accuracy Score: 0.383
Root Mean Square Error: 1.6774981371077584



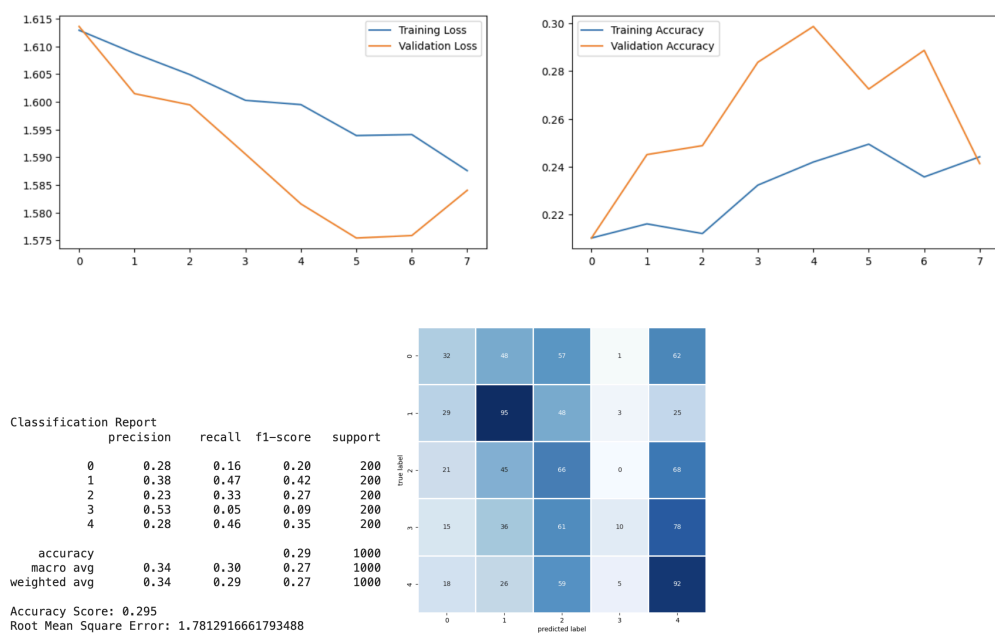
5. Experiment 5: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



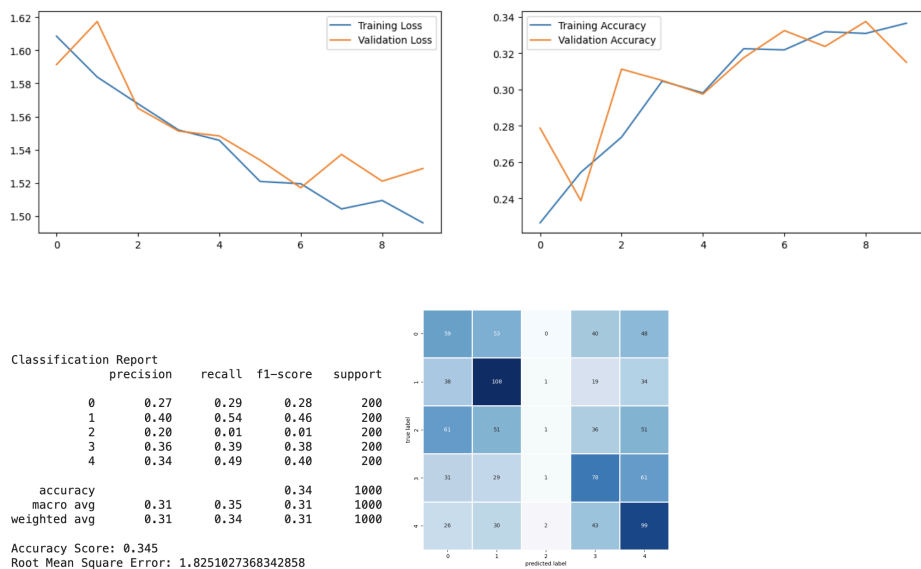
6. Experiment 6: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



7. Experiment 7: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix

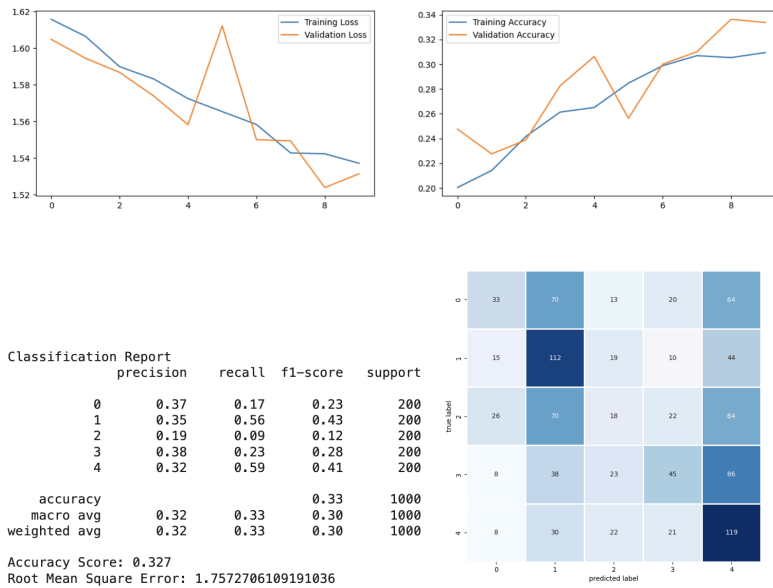


8. Experiment 8: Training & Validation Accuracy/Loss, Classification Report, Confusion Matrix



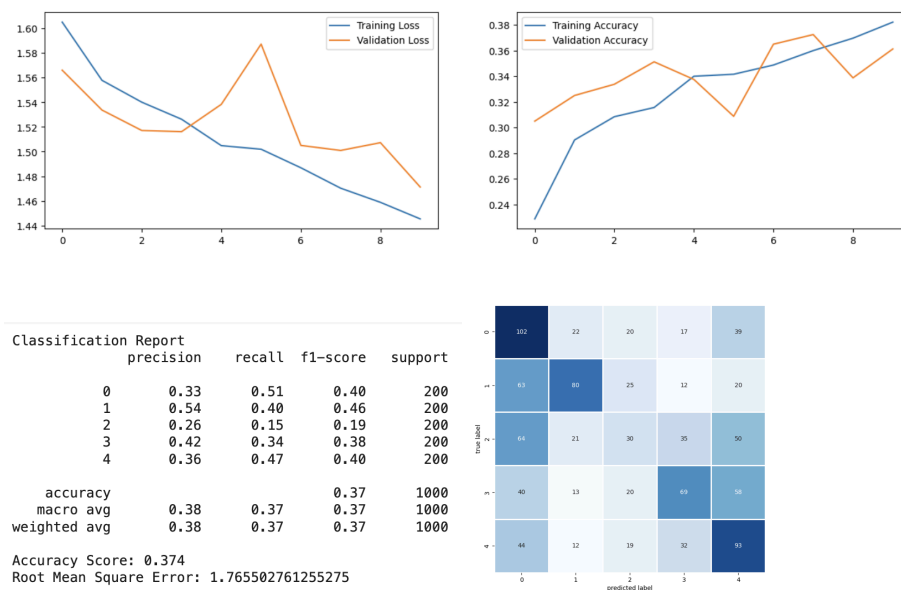
9. Experiment 9: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



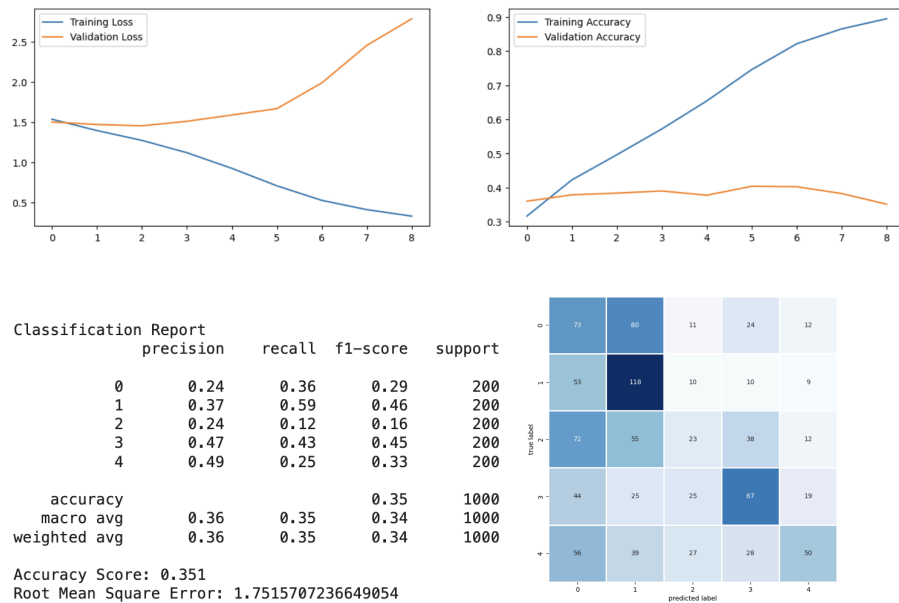
10. Experiment 10: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



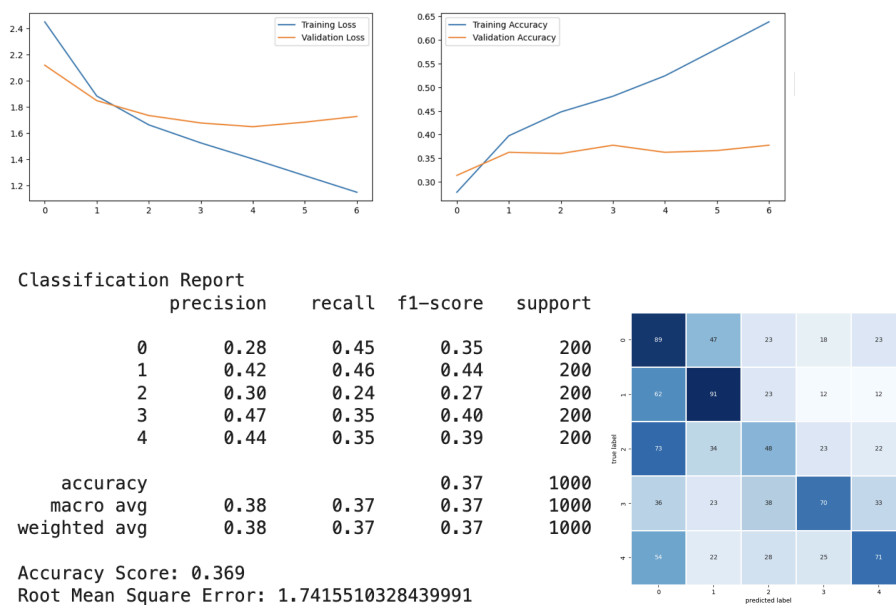
11. Experiment 11: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



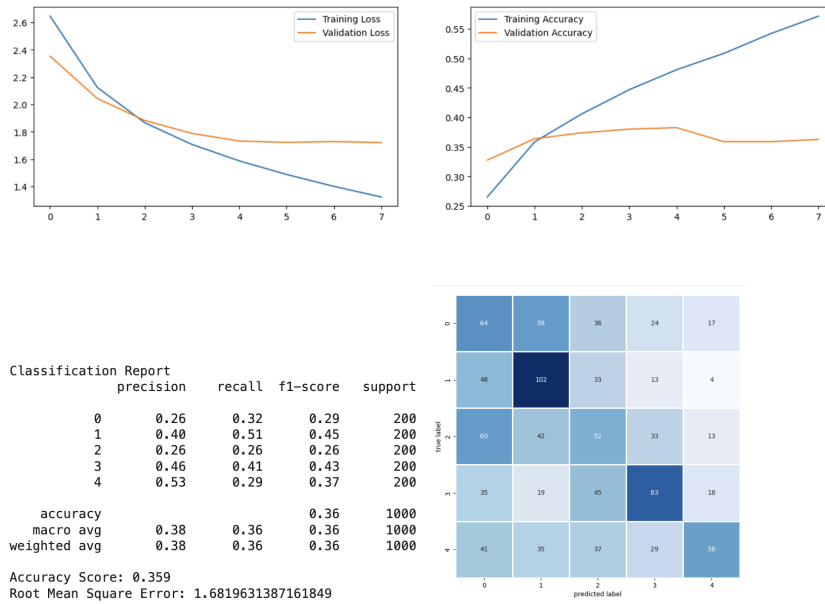
12. Experiment 12: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



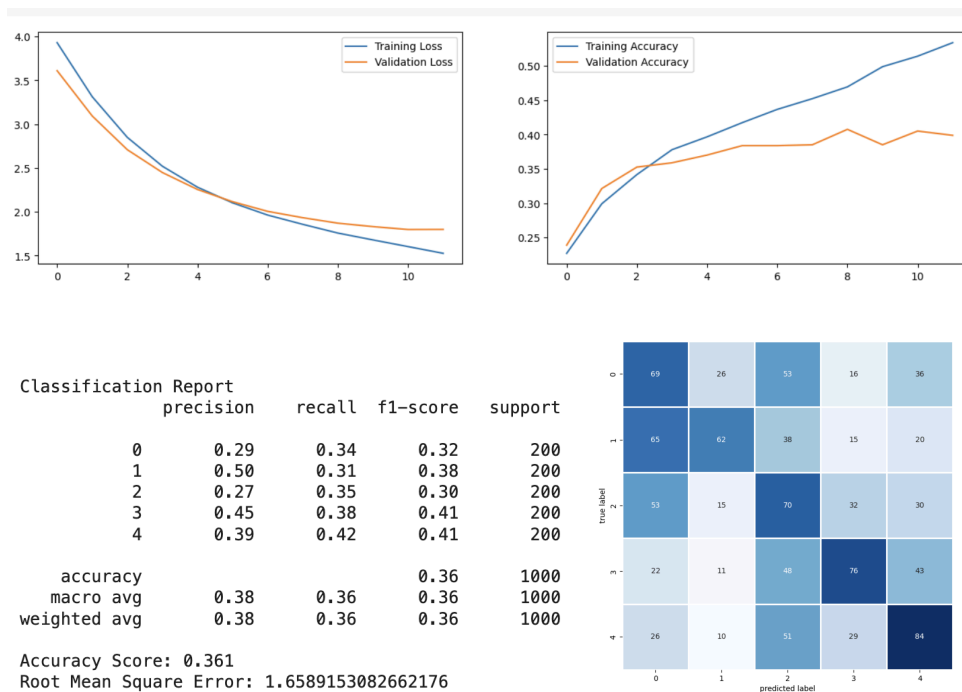
13. Experiment 13: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



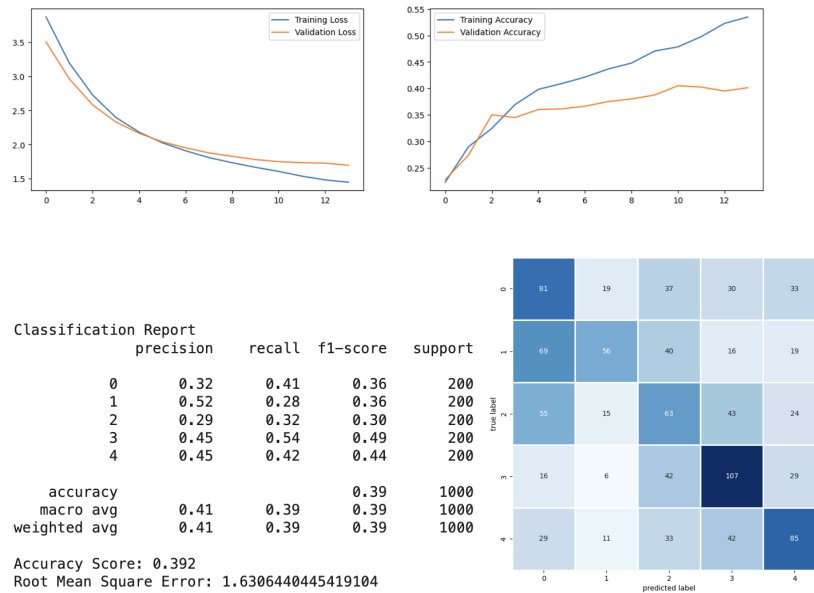
14. Experiment 14: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



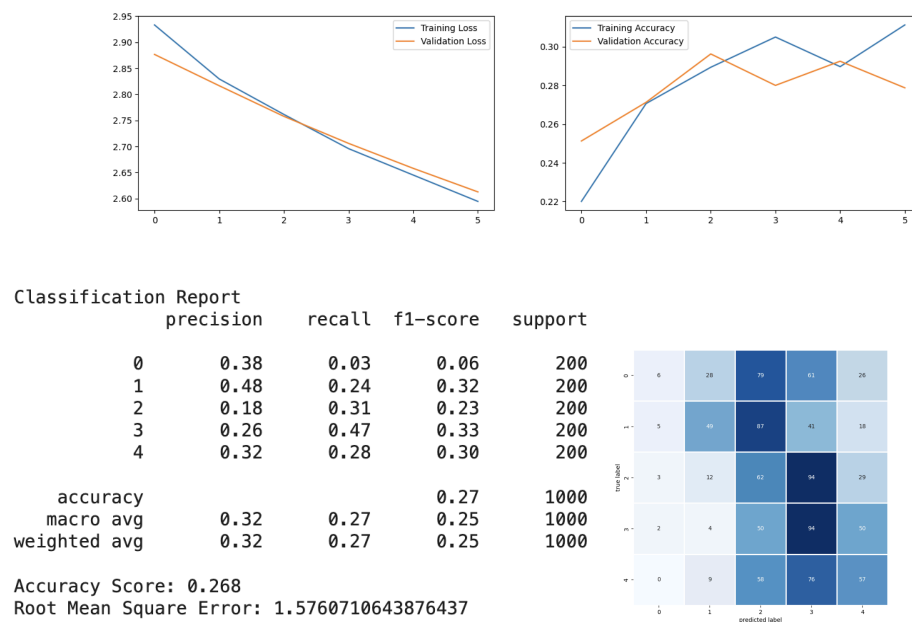
15. Experiment 15: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



16. Experiment 16: Training & Validation Accuracy/Loss, Classification Report, Confusion

Matrix



References

Ali, S. (2021, May 4). A 2021 guide to improving CNNs: Optimizers Adam vs. SGD. Medium.

Retrieved March 15, 2025, from

<https://medium.com/geekculture/a-2021-guide-to-improving-cnns-optimizers-adam-vs-sgd-495848ac6008>

DataCamp. (2024, December 9). Complete guide to data augmentation. Retrieved March 15, 2025, from <https://www.datacamp.com/tutorial/complete-guide-data-augmentation>

Niten. (n.d.). Face Shape Dataset [Dataset]. Kaggle.

<https://www.kaggle.com/datasets/niten19/face-shape-dataset/data>

Pratch-yani. (2020, November 5). Face Shape Classification using CNN [Computer software].

GitHub. Retrieved from <https://github.com/Pratch-yani/Face-Shape-Classification-using-CNN>